Tesla Stock Price Prediction

Using Multimodal Deep Learning

Updated Detailed Project Report

Includes model architectures, training settings, retrained baseline results, multimodal training options, and project scope/limitations.

UTS 49275 Neural Networks and Fuzzy Logic

Generated: 2026-04-27 17:15

Executive Summary

Project Aim

This project predicts TSLA stock movement using a returns-based forecasting pipeline. Instead of predicting raw closing prices directly, models predict future percentage returns and reconstruct prices afterward.

The key research question is whether a dedicated multimodal fusion architecture improves next-day price prediction compared with simpler standalone models trained on the same engineered feature set.

What Changed in the Latest Version

- The multimodal model now supports three training options: current setting, no sentiment, and sentiment without cross-attention.
- Regression is prioritized for MAE-focused evaluation with regression_weight = 1.0 and classification_weight = 0.1.
- Multi-day prediction is treated as a secondary objective with multi_day_weight = 0.05.
- All neural training is configured for 100 epochs because convergence was observed around epoch 50.

Scope and Limitations

Project Scope

The project is an educational financial sequence-modelling pipeline, not a production trading system. It focuses on model design, reproducibility, evaluation, and interpretability for next-day Tesla stock forecasting.

The primary metric is MAE on reconstructed next-day price. RMSE, MAPE, direction accuracy, and multi-day forecasts are secondary diagnostics.

Limitations

- Stock returns are noisy, non-stationary, and strongly affected by news/events not captured in OHLCV history.
- Live RSS sentiment is not true historical sentiment; long historical runs mostly rely on synthetic sentiment or neutral-filled current RSS data.
- A fixed chronological split is useful but weaker than walk-forward validation across multiple market regimes.
- Direction accuracy is a secondary three-class task and can be below 50% even when price MAE improves.
- The XGBoost model is saved as a pickle artifact; incompatible XGBoost native versions can crash while loading, so Streamlit skips live XGBoost loading.

Data and Feature Engineering

Data Sources

- Stock symbol: TSLA.
- Historical start date: 2010-06-29.
- Market context enabled: True; market context cache enabled: True.
- Sentiment real-data fetch default: False; Streamlit real sentiment default: False.

Feature Groups

- Price/OHLCV: open, high, low, close, volume, returns, log returns, and price range features.
- Technical indicators: SMA, EMA, RSI, MACD, Bollinger Bands, ATR, OBV, VWAP, stochastic oscillator, momentum, volatility, candlestick bodies/shadows, and gaps.
- Market context: SPY close/return, TSLA-vs-SPY alpha, and VIX.
- Calendar context: day of week, month sine/cosine, month end, and quarter end.
- Sentiment: compound, positive, negative, neutral, news count, and lagged sentiment values.

Training Settings and Configuration

Core Configuration

- Sequence length: 60 trading days.
- Prediction horizons: [1, 3, 5, 7].
- Direction neutral threshold: +/-0.50%.
- Batch size: 32.
- Learning rate: 0.0001.
- Epochs: 100.
- Chronological split: train 80%, validation 10%, test 10%.

Loss Priorities

- 1-day regression loss weight: 1.0.
- Direction classification loss weight: 0.1.
- Multi-day regression loss weight: 0.05.
- The objective is intentionally MAE/regression-oriented; classification and multi-day forecasts are auxiliary.

Active Saved Fusion Model

- Active fusion training mode: sentiment-no-cross-attention
- Uses sentiment branch: True
- Uses cross-attention: False
- Price/technical feature count: 59
- Sentiment feature count: 7
- Sequence length: 60 trading days
- Prediction horizons: [1, 3, 5, 7]

Pipeline Architecture

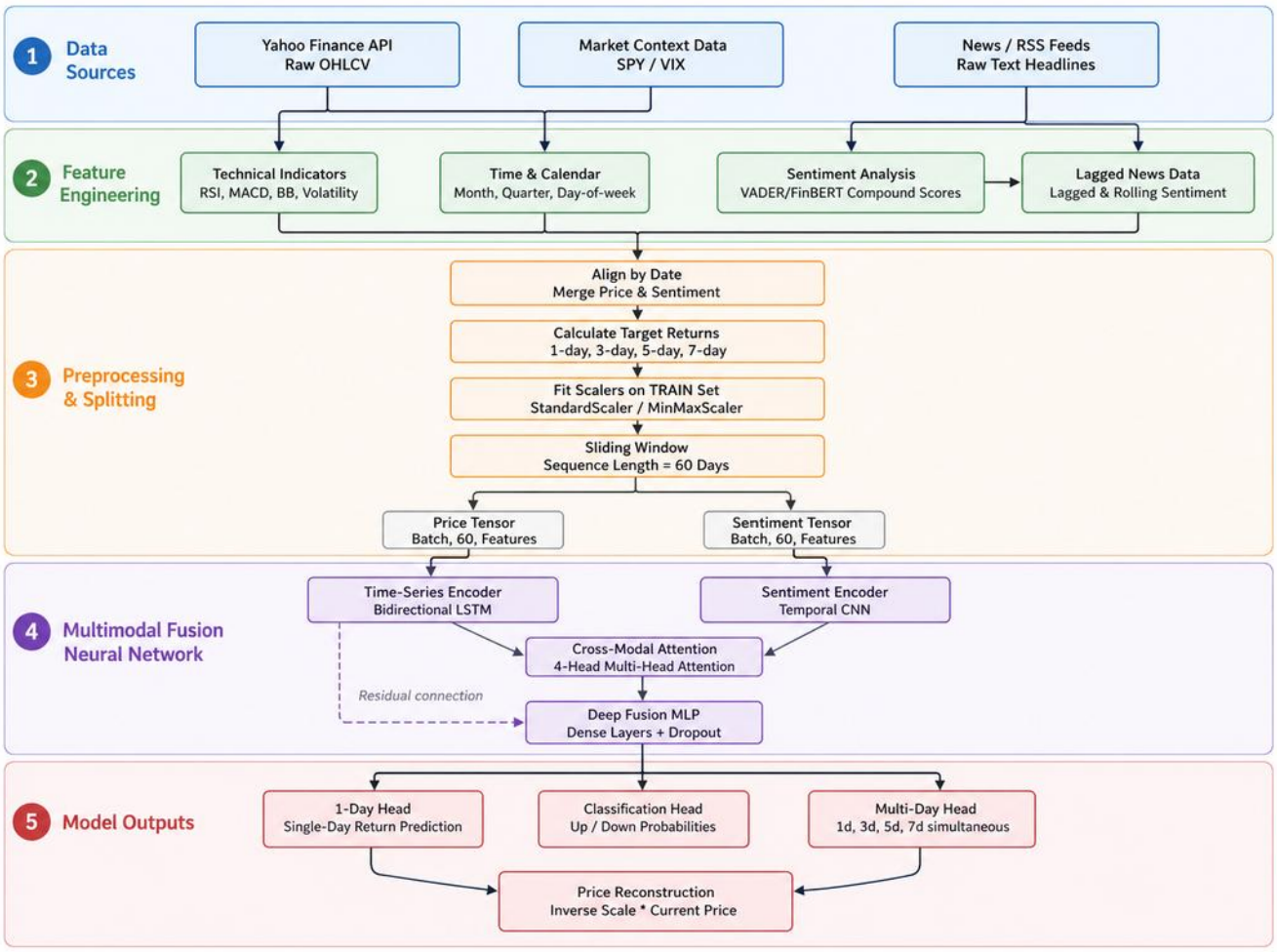

Figure 1. End-to-end workflow from TSLA data and sentiment features through preprocessing, fusion, and prediction heads.

Architecture: Multimodal Fusion Model

Input Structure

The multimodal model uses two streams: a price/technical stream and a sentiment stream. This differs from the standalone models, which concatenate all features into a single input tensor.

The active saved model currently uses sentiment and cross-attention according to preprocessing metadata.

Architecture Details

- Time-series encoder: LSTM-based encoder with hidden size 128, 2 layers, dropout 0.2, bidirectional = True.
- Sentiment encoder: temporal CNN encoder that maps sentiment sequences into a 256-dimensional representation.
- Cross-modal attention: optional multi-head attention where the time-series sequence attends to the encoded sentiment representation.
- Fusion MLP: hidden size 256 with dropout 0.3.
- Output heads: 1-day return regression, multi-day return regression, and 3-class direction classification.

Training Options

- current: sentiment branch enabled and cross-attention enabled.
- no-sentiment: sentiment branch disabled; model uses price, technical, market, and calendar features only.
- sentiment-no-cross-attention: sentiment branch enabled but cross-attention disabled; streams are fused by concatenation only.

Architecture: Standalone Baseline Models

Shared Baseline Input

The standalone models use the same preprocessing pipeline, but they concatenate price/technical and sentiment features into one sequence tensor. They are therefore not OHLCV-only baselines; they are single-stream baselines using the engineered feature set.

LSTM Regressor

- Input projection: Linear -> LayerNorm -> ReLU -> Dropout.
- Sequence model: 2-layer bidirectional LSTM with hidden size 128 and dropout 0.2.
- Output: concatenates forward/backward sequence representations, then Dense -> ReLU -> Dropout -> Dense(1).

GRU Regressor

- Input projection: Linear -> LayerNorm -> ReLU -> Dropout.
- Sequence model: 2-layer bidirectional GRU with hidden size 128 and dropout 0.2.
- Output: concatenates forward/backward sequence representations, then Dense -> ReLU -> Dropout -> Dense(1).

Transformer Regressor

- Input projection to d_model = 128.
- Sinusoidal positional encoding.
- Transformer encoder with 2 layers, 4 attention heads, feed-forward size 256, and dropout 0.1.
- Output uses the final token embedding followed by Dense -> ReLU -> Dropout -> Dense(1).

XGBoost Regressor

- The 60-day sequence is flattened into a tabular feature vector.
- XGBRegressor settings: 400 estimators, max_depth 4, learning_rate 0.05, subsample 0.8, random_state 42, n_jobs 1.
- This baseline tests whether a tree-based tabular learner can compete with neural sequence models on engineered features.

Evaluation Design and Retrained Results

Evaluation Design

The primary reported metric is MAE after reconstructing next-day prices from predicted returns. RMSE and MAPE are also reported for interpretability. Direction accuracy is secondary because the model is optimized mainly for regression.

The comparison is reasonable when framed as single-stream baselines versus a two-stream multimodal fusion architecture. It is not a comparison between OHLCV-only baselines and sentiment-aware multimodal modelling.

Saved Retrained Baseline Results

- LSTM: RMSE $12.72, MAE $9.70, MAPE 2.81%, Direction Accuracy 50.1%
- GRU: RMSE $12.72, MAE $9.70, MAPE 2.81%, Direction Accuracy 50.1%
- Transformer: RMSE $12.72, MAE $9.70, MAPE 2.81%, Direction Accuracy 50.1%
- XGBoost: RMSE $13.22, MAE $10.22, MAPE 2.94%, Direction Accuracy 49.9%

Interpretation

The latest saved baseline results show LSTM, GRU, and Transformer tied on MAE at $9.70, while XGBoost is slightly worse at $10.22. The exact tie among three neural models should be treated cautiously and checked against training logs or prediction distributions.

If the multimodal fusion model underperforms these baselines, the result is still academically meaningful: the available sentiment signal may not add enough information to offset the additional model complexity.

Training History

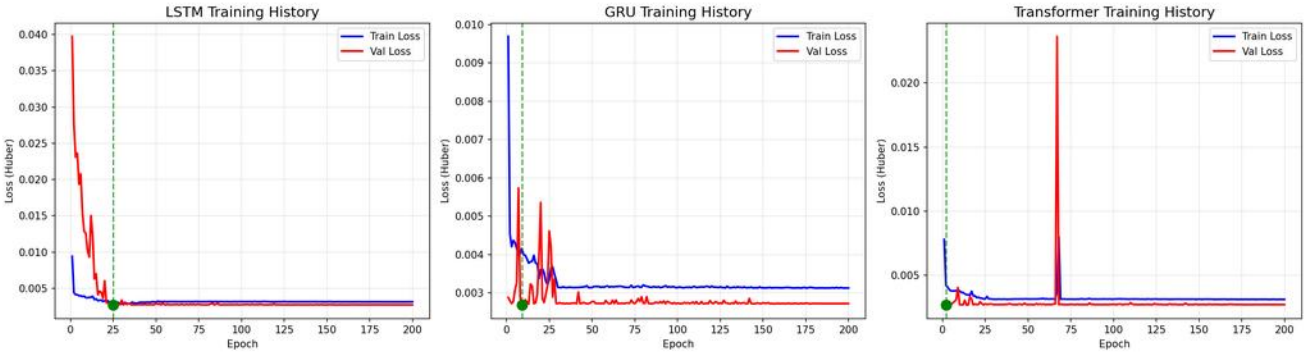

Figure 2. Saved training history visualisation for the multimodal project.

Dashboard and Deployment Notes

Streamlit Usage

Streamlit loads the active fusion model from models/best_model.pt and reconstructs the architecture using preprocessing_metadata.pkl.

Because XGBoost pickle loading caused native crashes with incompatible library versions, Streamlit skips live XGBoost artifact loading while still displaying saved XGBoost metrics from CSV.

Reproducibility Notes

- Use Python 3.11 or 3.12 with requirements-stable.txt for the Streamlit app.
- Use run_streamlit_safe.sh to disable Streamlit file watching and reduce native crash risk.
- Each new training run overwrites active artifacts in models/, so copy artifacts into experiment folders before training another mode.

Recommendations and Conclusion

Recommended Next Steps

- Save fusion test metrics to a CSV after every training run so the report can compare fusion modes directly.
- Run the three fusion ablations: current, no-sentiment, and sentiment-no-cross-attention.
- Select best_model.pt by validation 1-day regression loss or validation MAE, because MAE is the primary evaluation metric.
- Add walk-forward validation for a stronger time-series evaluation.
- Replace synthetic/current RSS sentiment with a genuine historical news source if the project scope allows.

Conclusion

The current project setting is suitable for coursework because it includes a full ML workflow, defensible baselines, multimodal architecture, ablation options, and a clear scope statement. The most important caveat is that the sentiment signal is limited, so standalone models outperforming the multimodal model is a valid and explainable result rather than a project failure.

References

Core References

- Hochreiter, S. and Schmidhuber, J. Long Short-Term Memory. Neural Computation, 1997.
- Cho, K. et al. Learning Phrase Representations using RNN Encoder-Decoder for Statistical Machine Translation, 2014.
- Vaswani, A. et al. Attention Is All You Need, 2017.
- Chen, T. and Guestrin, C. XGBoost: A Scalable Tree Boosting System, 2016.
- Araci, D. FinBERT: Financial Sentiment Analysis with Pre-trained Language Models, 2019.
- Project source files: config.py, train.py, model_comparison.py, src/data, src/features, src/models, app/streamlit_app.py.